

Question Bank

1.Explain Python Features.

- Free and Open Source.
- Easy to code
- Easy to Read
- Object-Oriented Language
- GUI Programming Support
- High-Level Language
- Extensible feature
- Easy to Debug
- Python is a Portable language
- Python is an Integrated language
- Interpreted Language
- Large standard library
- Dynamically Typed Language
- Frontend and backend development
- Allocating Memory Dynamically

2.List Out Python Applications and Explain in Brief.

1. Web Development

It is one of the most astonishing applications of Python. This is because Python comes up with a wide range of frameworks like Django, Flask, Bottle, and a lot more that provide ease to developers. Furthermore, Python has inbuilt libraries and tools which make the web development process completely effortless. Use of Python for web development also offers:

- Amazing visualization
- Convenience in development
- Enhanced security
- Fast development processes

2. Machine Learning and Artificial Intelligence

Machine Learning and Artificial Intelligence are the hottest subjects right now. Python along with its inbuilt libraries and tools facilitate the development of AI and ML algorithms. Further, it offers simple, concise, and readable code which makes it easier for developers to write complex algorithms and provide a versatile flow. Some of the inbuilt libraries and tools that enhance AI and ML processes are:

- Numpy for complex data analysis

- Keras for Machine learning
- SciPy for technical computing
- Seaborn for data visualization

3. Data Science

Data science involves data collection, data sorting, data analysis, and data visualization. Python provides amazing functionality to tackle statistics and complex mathematical calculations. The presence of in-built libraries provides convenience to data science professionals. Some of the popular libraries that provide ease in the data science process are TensorFlow, Pandas, and Socket learning. These libraries provide an ecosystem for fine-tuning data models, data preprocessing, and performing complex data analysis.

4. Game Development

With the rapidly growing gaming industry Python has proved to be an exceptional option for game development. Popular games like Pirates of the Caribbean, Bridge commander, and Battlefield 2 use Python programming for a wide range of functionalities and addons. The presence of popular 2D and 3D gaming libraries like pygame, panda3D, and Cocos2D make the game development process completely effortless.

5. Audio and Visual Applications

Audio and video applications are undoubtedly the most amazing feature of Python. Python is equipped with a lot of tools and libraries to accomplish your task flawlessly. Applications that are coded in Python include popular ones like Netflix, Spotify, and YouTube. This can be handled by libraries like

- Dejavu
- Pyo
- Mingus
- SciPy
- OpenCV

6. Software Development

Python is just the perfect option for software development. Popular applications like Google, Netflix, and Reddit all use Python. This language offers amazing features like:

- Platform independence
- Inbuilt libraries and frameworks to provide ease of development.
- Enhanced code reusability and readability
- High compatibility

Apart from these Python offers enhanced features to work with rapidly growing technologies like Machine learning and Artificial intelligence. All these embedded features make it a popular choice for software development.

7. CAD Applications

CAD refers to computer-aided design; it is the process of creating 3D and 2D models digitally. This application has replaced manual draft and is used by architects, product designers, and construction managers to design products with extremely high consistency. Python is embedded with amazing applications like Blender, FreeCAD, open cascade, and a lot more to efficiently design products. These provide enhanced features like technical drawing, dynamic system development, recordings, file export, and import.

8. Business Applications

Python offers excellent security and scalability features that make it perfect for delivering high-performance business applications. It has inbuilt libraries and tools like:

Odoo is business management software that provides you with an automated solution for your business process.

Tryton is easy-to-use open-source business software. It has fully integrated features like financial accounting, sales, CRM, purchasing, shipping, and the list goes on.

All these distinguishing features make it fit for creating business applications.

9. Desktop GUI

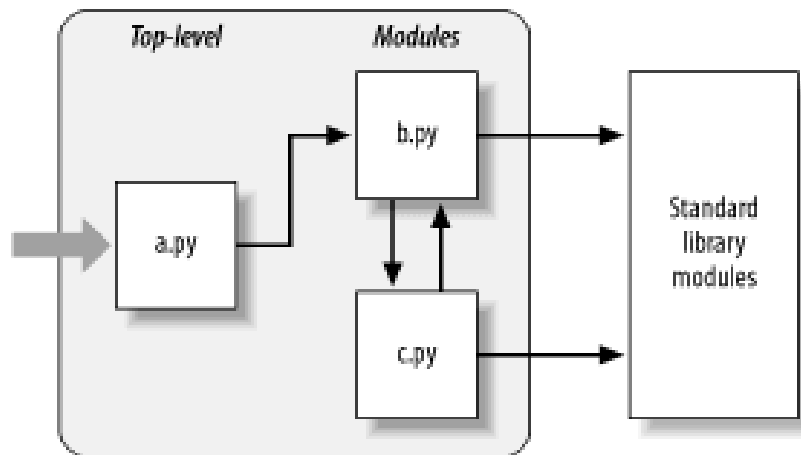
Python is an interactive programming language that helps developers to create GUIs easily and efficiently. It has a huge list of inbuilt tools like PyQT, kivy, wxWidgets, and many other libraries like them to build a fully functional GUI in an extremely secure and efficient manner.

10. Web Scraping Application

Web scraping is an automated process used to extract information from websites in an easier and faster way. The information is used by researchers, organizations, and analysts for a wide variety of tasks. Python has a wide range of features that make it suitable for web scraping some of them are:

- A concise syntax that enhances the readability and saves your time.
- A wide range of libraries and tools like pandas, matplotlib, and Selenium makes the web scraping process easy and efficient.
- Easy to use and understand.

3.Explain Basic Structure Of Python Program in Brief.



4.What is Keyword?Give Example Of Keyword.List Out All Keywords Present in Python3.

Keywords in Python are reserved words that can not be used as a variable name, function name, or any other identifier.

List of all keywords in Python

Keywords in Python				
All keywords has special meaning and purpose in python. Here is list of all keywords in python:				
False	None	True	and	as
assert	break	class	continue	def
del	elif	else	except	finally
for	from	global	if	import
in	is	lambda	nonlocal	not
or	pass	raise	return	try
while	with	yield		

5. What is Identifier ? Explain Identifier With Example.

Identifiers are the name given to variables, classes, methods, etc. For example,

```
language = 'Python'
```

Here, language is a variable (an identifier) which holds the value 'Python'.

We cannot use keywords as variable names as they are reserved names that are built-in to Python. For example,

```
continue = 'Python'
```

The above code is wrong because we have used continue as a variable name.

6.List out Builtin Data types In Python. Explain With Example.

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

- Numeric data types: int, float, complex
- String data types: str
- Sequence types: list, tuple, range
- Binary types: bytes, bytearray, memoryview
- Mapping data type: dict
- Boolean type: bool
- Set data types: set, frozenset

7/8 .How To Create Variable In Python? Explain With Example.

Rules for creating variables in Python

- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable names are case-sensitive (name, Name and NAME are three different variables).
- The reserved words(keywords) cannot be used naming the variable.

Example:- x=2

```
print(x)
```

```
print(type(x))
```

9 .Explain Many Values To Multiple Variables And One Value To Multiple Variables.

Assign Value to Multiple Variables

Python allows you to assign values to multiple variables in one line:

Example :

```
x, y, z = "Orange", "Banana", "Cherry"
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

Output :-

Orange

Banana

Cherry

And you can assign the same value to multiple variables in one line:

Example:-

```
x = y = z = "Orange"
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

Output :-

Orange

Orange

Orange

10 .Explain Type Casting In Python.Explain About Implicit Type Casting And Explicit Type Casting.

Type Conversion is the conversion of an object from one data type to another data type.

Implicit Type Conversion:- In this, methods, Python converts data type into another data type automatically. In this process, users don't have to involve in this process.

Implicit Type Conversion is automatically performed by the Python interpreter.

Python avoids the loss of data in Implicit Type Conversion.

Explicit Type Casting :-In this method, Python need user involvement to convert the variable data type into certain data type in order to the operation required.

Explicit Type Conversion is also called Type Casting, the data types of objects are converted using predefined functions by the user.

In Type Casting, loss of data may occur as we enforce the object to a specific data type.

11. How to take multiple Input In Python give example.

Python user can take multiple values or inputs in one line by two methods.

Using split() method :- This function helps in getting multiple inputs from users. It breaks the given input by the specified separator. If a separator is not provided then any white space is a separator. Generally, users use a split() method to split a Python string but one can use it in taking multiple inputs.

Syntax :

```
input().split(separator, maxsplit)
```

Example :

```
x, y = input("Enter two values: ").split()
```

```
print("Number of boys: ", x)
```

```
print("Number of girls: ", y)
```

```
print()
```

Using List comprehension :- List comprehension is an elegant way to define and create list in Python. We can create lists just like mathematical statements in one line only. It is also used in getting multiple inputs from a user.

Example :-

```
x, y = [int(x) for x in input("Enter two values: ").split()]
```

```
print("First Number is: ", x)
```

```
print("Second Number is: ", y)
```

```
print()
```

12. What Is Operator ? List Out All Operators. Explain It With Example.

Python Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations.

Types of Python Operators

1. Arithmetic operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators

6. Special Operators

i) Arithmetic Operators :- Arithmetic operators are used to performing mathematical operations like addition, subtraction, multiplication, and division.

Operators	Meaning	Example	Result
+	Addition	$4 + 2$	6
-	Subtraction	$4 - 2$	2
*	Multiplication	$4 * 2$	8
/	Division	$4 / 2$	2
%	Modulus operator to get remainder in integer division	$5 \% 2$	1
**	Exponent	$5^{**}2 = 5^2$	25
//	Integer Division/ Floor Division	$5//2$ $-5//2$	2 -3

ii) Assignment Operators :- Assignment operators are used to assign values to variables. For example,

assign 5 to x

var x = 5

Operator	Example	Equivalent Expression (m=15)	Result
=	$y = a+b$	$y = 10 + 20$	30
+=	$m += 10$	$m = m + 10$	25
-=	$m -= 10$	$m = m - 10$	5
*=	$m *= 10$	$m = m * 10$	150
/=	$m /= 10$	$m = m / 10$	1.5
%=	$m \% = 10$	$m = m \% 10$	5
=	$m ** = 2$	$m = m^{}2$ or $m = m^2$	225
//=	$m //= 10$	$m = m // 10$	1

iii) Comparison Operators :- Comparison operators compare two values/variables and return a boolean result: True or False.

For example,

a = 5

b = 2

```
print (a > b)  # True
```

Operators	Meaning	Example	Result
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True
==	Equal to	5==2	False
!=	Not equal to	5!=2	True

iv) Logical Operators :-

Logical operators are used to check whether an expression is True or False. They are used in decision-making. For example,

a = 5

b = 6

```
print((a > 2) and (b >= 6))  # True
```

Operator	Meaning	Example	Result
and	Logical and	(5<2) and (5>3)	False
or	Logical or	(5<2) or (5>3)	True
not	Logical not	not (5<2)	True

v) Bitwise operators :- Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name.

For example, 2 is 10 in binary and 7 is 111.

Operator	Meaning
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR / Bitwise XOR
~	Bitwise inversion (one's complement)
<<	Shifts the bits to left / Bitwise Left Shift
>>	Shifts the bits to right / Bitwise Right Shift

vi) Special operators :- Python language offers some special types of operators like the identity operator and the membership operator.

- Identity operators :- In Python, is and is not are used to check if two values are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Example	Description
is	x is y	Returns True if both variables are the same object
is not	x is not y	Returns True if both variables are not the same object

SCALER
Topics

- Membership operators :- In Python, in and not in are the membership operators. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary). In a dictionary we can only test for presence of key, not the value.

Operator	Example	Description
in	5 in {1, 2, 3, 4, 5}	It returns true if it finds the corresponding value in a specified sequence and false otherwise.
not in	5 not in {1, 2, 3, 4}	It returns true if it does not find the corresponding value in a given sequence and true otherwise.

SCALER
Topics

13. Can We Concat String And Number Using + Operator ?Yes Or No? If No Then Specify.

Python supports string concatenation using the + operator. In most other programming languages, if we concatenate a string with an integer (or any other primitive data types), the language takes care of

converting them to a string and then concatenates it. However, in Python, if you try to concatenate a string with an integer using the + operator, you will get a runtime error.

Example

```
current_year_message = 'Year is '  
  
current_year = 2018  
  
print(current_year_message + current_year)
```

However, when we run this code we get the following runtime error: - TypeError: can only concatenate str (not "int") to str.

Solution :-

1. Using the str() Function :- We can pass an int to the str() function it will be converted to a str:
 - a. `print(current_year_message + str(current_year))`
2. Using the % Interpolation Operator :- We can pass values to a conversion specification with printf-style String Formatting:
 - a. `print("%s%s" % (current_year_message, current_year))`
3. Using the str.format() function :- We can also use the str.format() function for concatenation of string and integer.
 - a. `print("{} {}".format(current_year_message, current_year))`
4. Using f-strings :- If you are using Python 3.6 or higher versions, you can use f-strings, too.
`print(f'{current_year_message}{current_year}')`

14. Explain Separator And End argument With Example.

sep parameter :-

The separator between the arguments to print() function in Python is space by default (softspace feature) , which can be modified and can be made to any character, integer or string as per our choice. The 'sep' parameter is used to achieve the same, it is found only in python 3.x or later. It is also used for formatting the output strings.

Example :-

```
#code for disabling the softspace feature
```

```
print('G','F','G', sep="")
```

```
#for formatting a date
```

```
print('09','12','2016', sep='-')
```

```
#another example
```

```
print('pratik','geeksforgeeks', sep='@')
```

Output:

GFG

09-12-2016

pratik@geeksforgeeks

Chapter :- 2

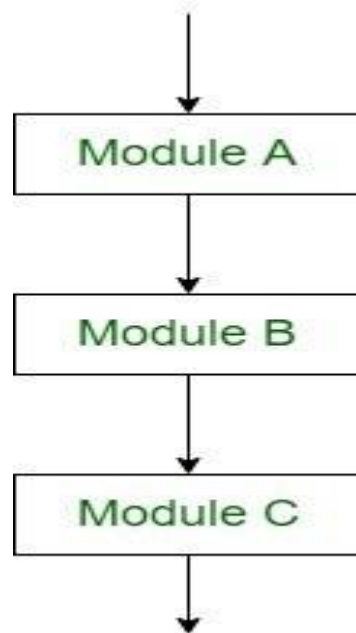
1. What is Control Structure ? Explain types of control structure with example.

Control Structures are just a way to specify flow of control in programs. Any algorithm or program can be more clear and understood if they use self-contained modules called as logic or control structures. It basically analyzes and chooses in which direction a program flows based on certain parameters or conditions. There are three basic types of logic, or flow of control, known as:

- Sequence logic
- Selection logic
- Iteration logic

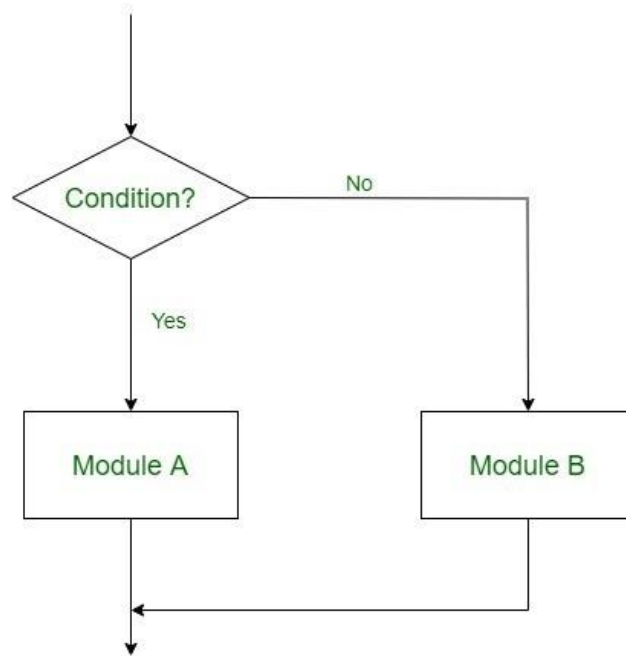
Sequential Logic (Sequential Flow):-

Sequential logic as the name suggests follows a serial or sequential flow in which the flow depends on the series of instructions given to the computer. Unless new instructions are given, the modules are executed in the obvious sequence. The sequences may be given, by means of numbered steps explicitly. Also, implicitly follows the order in which modules are written. Most of the processing, even some complex problems, will generally follow this elementary flow pattern.



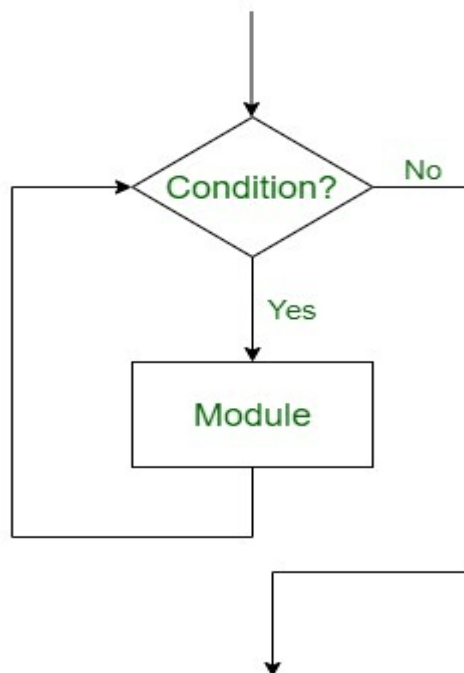
Selection Logic (Conditional Flow) :-

Selection Logic simply involves a number of conditions or parameters which decides one out of several written modules. The structures which use these type of logic are known as Conditional Structures. These structures can be of three types:



Iteration Logic (Repetitive Flow) :-

The Iteration logic employs a loop which involves a repeat statement followed by a module known as the body of a loop.



2. Explain if and if-else statements with example.

if statement

if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statement is executed otherwise not.

Syntax:

if condition:

 # Statements to execute if condition is true

statements 2

Example:-

number = 10

check if number is greater than 0

if number > 0:

 print('Number is positive.')

print('The if statement is easy.')

Output:-

Number is positive.

The if statement is easy.

if-else statement :-

The block of code following the else statement is executed as the condition present in the if statement is false after calling the statement which is not in block(without spaces).

Syntax:-

if condition:

 # block of code if condition is True

else:

 # block of code if condition is False

Example:-

```
number = 10  
  
if number > 0:  
    print('Positive number')  
  
else:  
    print('Negative number')  
  
print('This statement is always executed')
```

Output :-

```
Positive number  
  
This statement is always executed
```

3. Explain nested if-else and if-elif-else statements with example.

nested-if

A nested if is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, Python allows us to nest if statements within if statements. i.e, we can place an if statement inside another if statement.

Syntax:

```
if (condition1):  
    # Executes when condition1 is true  
  
    if (condition2):  
        # Executes when condition2 is true  
  
    # if Block is end here  
  
# if Block is end here
```

Example :-

```
number = 5  
  
# outer if statement  
  
if (number >= 0):
```



```
# inner if statement

if number == 0:

    print('Number is 0') # inner else statement

else:

    print('Number is positive')

# outer else statement

else:

    print('Number is negative')
```

Output: Number is positive

if-elif-else ladder

Here, a user can decide among multiple options. The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

Syntax:

```
if (condition):

    statement

elif (condition):

    statement

.

.else:

    statement
```

Example:-

```
number = 0

if number > 0:

    print("Positive number")
```

```
elif number == 0:

    print('Zero')

else:

    print('Negative number')

print('This statement is always executed')
```

Output :-

Zero

This statement is always executed

4. Explain switch statement with example.

Switch Case is a selection control statement. The switch expression is evaluated once. The value of the expression is compared with the values of each case; if there is a match, the associated block of code is executed.

Example :-

```
def number_to_string(argument):

    match argument:

        case 0:

            return "zero"

        case 1:

            return "one"

        case 2:

            return "two"

        case default:

            return "something"

if __name__ == "__main__":

    argument = 0

    number_to_string(argument)
```

5. Explain for loop with example.

For loops are used for sequential traversal. For example: traversing a list or string or array etc. In Python, there is no C style for loop, i.e., for (i=0; i<n; i++). There is "for in" loop which is similar to for each loop in other languages. Let us learn how to use for in loop for sequential traversals.

Syntax:

for iterator_var in sequence:

 statements(s)

It can be used to iterate over a range and iterators.

Example :-

n = 4

for i in range(0, n):

 print(i)

Output :-

0

1

2

3

6. Explain while loop with example.

While Loop is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

Syntax:

while expression:

 statement(s)

Example :-

count = 0

while (count < 3):

 count = count + 1

```
print("Hello Geek")
```

Output :-

Hello Geek

Hello Geek

Hello Geek

7. Explain nested loops with example.

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop":

Syntax:

Outer_loop Expression:

 Inner_loop Expression:

 Statement inside inner_loop

Example :-

```
x = [1, 2]
```

```
y = [4, 5]
```

```
for i in x:
```

```
    for j in y:
```

```
        print(i, j)
```

Output :-

1 4

1 5

2 4

2 5

8. Explain for loop with break, continue and pass statement in brief.

Break statement

The break statement is used to terminate the loop or statement in which it is present. After that, the control will pass to the statements that are present after the break statement, if available. If the break statement is present in the nested loop, then it terminates only those loops which contains break statement.

Syntax:

break

Example :-

```
for i in range(5):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Output :-

0

1

3

Continue statement

Continue is also a loop control statement just like the break statement. continue statement is opposite to that of break statement, instead of terminating the loop, it forces to execute the next iteration of the loop.

As the name suggests the continue statement forces the loop to continue or execute the next iteration. When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.

Syntax:

continue

Example :-

```
for i in range(5):
```

```
    if i == 3:
```

```
        continue
```

```
print(i)
```

Output :-

0

1

2

4

Pass statement

As the name suggests pass statement simply does nothing. The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute. It is like null operation, as nothing will happen is it is executed. Pass statement can also be used for writing empty loops. Pass is also used for empty control statement, function and classes.

Syntax :-

```
pass
```

Example :-

```
if 1 + 2 == 3:
```

```
    print("Correct math")
```

```
    pass
```

```
    print("This will also be printed.")
```

Output :-

Correct math

This will also be printed.

9. Explain while loop with break, continue and pass statement in brief.

Break

program to find first 5 multiples of 6

```
i = 1
```

```
while (i<=10):
```

```
    print('6 * ',i), '=' ,6 * i
```

```
if i >= 5:
```

```
    break
```

```
    i = i + 1
```

Output :-

```
6 * 1 = 6
```

```
6 * 2 = 12
```

```
6 * 3 = 18
```

```
6 * 4 = 24
```

```
6 * 5 = 30
```

Continue

```
# program to print odd numbers from 1 to 10
```

```
num = 0
```

```
while num < 10:
```

```
    num += 1
```

```
    if (num % 2) == 0:
```

```
        continue
```

```
    print(num)
```

Output :-

```
1
```

```
3
```

```
5
```

```
7
```

```
9
```

Pass

```
number = 0
```

```
for number in range(10):  
    if number == 5:  
        pass # pass here  
    print('Number is ' + str(number))  
print('Out of loop')
```

Output :-

```
Number is 0  
Number is 1  
Number is 2  
Number is 3  
Number is 4  
Number is 5  
Number is 6  
Number is 7  
Number is 8  
Number is 9  
Out of loop
```

10. Explain range() function using start parameter.

The Python range() function returns a sequence of numbers, in a given range. The most common use of it is to iterate sequence on a sequence of numbers using Python loops.

Syntax of Python range() function

Syntax: range(start, stop, step)

Parameter:

- start: [optional] start value of the sequence
- stop: next value after the end value of the sequence

- step: [optional] integer value, denoting the difference between any two numbers in the sequence.
- Return: Returns a range type object.

Example :-

```
for i in range(5):
```

```
    print(i, end=" ")
```

```
print()
```

Output :-

```
0 1 2 3 4
```

11. Explain range() function using increment sequence with 3.

If we pass all three arguments,

the first argument is start

the second argument is stop

the third argument is step

The step argument specifies the incrementation between two numbers in the sequence.

Example :-

```
# numbers from 2 to 10 with increment 3 between numbers
```

```
numbers = range(2, 10, 3)
```

```
print(list(numbers)) # [2, 5, 8]
```

12. Explain short hand if-else with example.

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:

Example :-

```
a = 2
```

```
b = 330
```

```
print("A") if a > b else print("B")
```

Output :-

B

Chapter :- 3

1. what is list in python ?

Python Lists are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java). In simple language, a list is a collection of things, enclosed in [] and separated by commas.

The list is a sequence data type which is used to store the collection of data. Tuples and String are other types of sequence data types.

Example :-

```
# A list with 3 integers
```

```
numbers = [1, 2, 5]
```

```
print(numbers)
```

```
# Output: [1, 2, 5]
```

2. how to iterate items in the list?

List is equivalent to arrays in other languages, with the extra benefit of being dynamic in size. In Python, the list is a type of container in Data Structures, which is used to store multiple data at the same time. Unlike Sets, lists in Python are ordered and have a definite count.

There are multiple ways to iterate over a list in Python. Let's see all the different ways to iterate over a list in Python, and performance comparison between them.

Method #1: Using For loop

```
list = [1, 3, 5, 7, 9]
```

```
# Using for loop
```

```
for i in list:
```

```
    print(i)
```

Output :-

1

3

5

7

9

Method #2: For loop and range()

In case we want to use the traditional for loop which iterates from number x to number y.

Python3 code to iterate over a list

```
list = [1, 3, 5, 7, 9]
```

getting length of list

```
length = len(list)
```

Iterating the index

same as 'for i in range(len(list))'

```
for i in range(length):
```

```
    print(list[i])
```

Output :- same as method 1

Method #3: Using while loop

```
list = [1, 3, 5, 7, 9]
```

Getting length of list

```
length = len(list)
```

```
i = 0
```

Iterating using while loop

```
while i < length:
```

```
    print(list[i])
```

```
    i += 1
```

Method #4: Using list comprehension (Possibly the most concrete way).

```
list = [1, 3, 5, 7, 9]
```

```
# Using list comprehension
```

```
[print(i) for i in list]
```

Method #5: Using enumerate()

If we want to convert the list into an iterable list of tuples (or get the index based on a condition check, for example in linear search you might need to save the index of minimum element), you can use the `enumerate()` function.

```
list = [1, 3, 5, 7, 9]
```

```
# Using enumerate()
```

```
for i, val in enumerate(list):
```

```
    print (i, ", ", val)
```

Output :-

```
0,1
```

```
1,3
```

```
2,5
```

```
3,7
```

```
4,9
```

3. Is list is immutable? Justify your answer.

Unlike strings, lists are mutable. This means we can change an item in a list by accessing it directly as part of the assignment statement. Using the indexing operator (square brackets) on the left side of an assignment, we can update one of the list items.

Example :-

```
color = ["red", "blue", "green"]
```

```
print(color)
```

```
color[0] = "pink"
```

```
color[-1] = "orange"
```

```
print(color)
```

Output :-

```
['red', 'blue', 'green']
```

```
['pink', 'blue', 'orange']
```

4. What is the different between append() and extend() functions.?

Append

It adds an element at the end of the list. The argument passed in the append function is added as a single element at end of the list and the length of the list is increased by 1.

Syntax:

```
list_name.append(element)
```

This method appends each element of the iterable (tuple, string, or list) to the end of the list and increases the length of the list by the number of elements of the iterable passed as an argument.

```
list_name.extend(iterable)
```

Difference between Append and extend

append()	extend()
The element passed as an argument is appended to the end of the list.	Each element of the iterable passed as an argument gets appended to the end of the list.
An iterable passed as an argument appends without any change as a single element to the end of the list.	An iterable passed as an argument will append each of its elements to the end of the list.
Length of the list increases by 1.	Length of the list increases by the number of elements in the iterable.
Has a constant time complexity of $O(1)$.	Has a time complexity of $O(k)$ where k is the length of the iterable.

5. What is the difference between “remove” and “pop”?

remove() Method:

The remove() method removes the first matching value from the list.

Syntax:

```
list_name.remove(value)
```

pop() Method:

The pop() method like del deletes value at a particular index. But pop() method returns deleted value from the list. if no index value is given then it will pop last element of the list.

Syntax:

list_name.pop(index)

Difference between Remove and pop

Remove()	pop()
It is a method to delete value this method uses the value as a parameter.	pop() is a method this method also uses the index as a parameter to delete.
The remove() method doesn't return any value.	pop() returns deleted value.
It throws value error in case of value doesn't exist in the list.	It throws index error in case of an index doesn't exist in the list.

6. How to manipulate every element in a list with list comprehension.

A Python list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterate over each element in the Python list.

Python List comprehension provides a much more short syntax for creating a new list based on the values of an existing list.

Syntax of List Comprehension

newList = [expression(element) for element in oldList if condition]

Example :-

List = [character for character in [1, 2, 3]]

Displaying list

print(List)

Output :-

[1, 2, 3]

list = [i for i in range(11) if i % 2 == 0]

```
print(list)
```

Output :- [0, 4 , 6, 8, 10]

7. What do you mean by nested list. Give example.

Lists are useful data structures commonly used in Python programming. A nested list is a list of lists, or any list that has another list as an element (a sublist). They can be helpful if you want to create a matrix or need to store a sublist along with other data types.

Example :-

```
# creating a nested list
```

```
Nested_List = [10, 20, 30,['a', 'b', 'c'], 50]
```

```
# accessing the sublist by using the indexing method
```

```
Sub_List = Nested_List[3]
```

```
# accessing the second element of the sublist
```

```
data = Nested_List[3][1]
```

```
print("List inside the nested list: ", Sub_List)
```

```
print("Second element of the sublist: ", data)
```

Output :-

```
List inside the nested list: ['a', 'b', 'c']
```

```
Second element of the sublist: b
```

Example 2 :-

```
# Creating a nested list
```

```
Nested_list = [1, [2, 3], 4]
```

```
# adding 5 inside the sublist using the append() function
```

```
Nested_list[1].append(5)
```

```
print(Nested_list)
```

Output :-

```
[1, [2, 3, 5], 4]
```


Chapter :- 4

1. How to create user define function in python. Explain with example.

All the functions that are written by any us comes under the category of user defined functions. Below are the steps for writing user defined functions in Python.

- In Python, def keyword is used to declare user defined functions.
- An indented block of statements follows the function name and arguments which contains the body of the function.

Syntax:

```
def function_name():
```

```
    statements
```

```
    .
```

```
    .
```

Example :-

```
def fun():
```

```
    print("Inside function")
```

```
# Driver's code
```

```
# Calling function
```

```
fun()
```

Output :- inside function

Example 2:-

```
def evenOdd( x ):
```

```
    if (x % 2 == 0):
```

```
        print("even")
```

```
    else:
```

```
print("odd")
```

```
# Driver code
```

```
evenOdd(2)
```

```
evenOdd(3)
```

Output :-

```
Even
```

```
Odd
```

2. How to call a function in python .Explain with example.

To call a function, you write out the function name with suitable parameters followed by a colon.

The syntax for calling a function looks like this:

```
function_name()
```

Make sure you don't specify the function call inside the function block. It won't work that way because the call will be treated as a part of the function to run.

Example :-

```
def greet():
```

```
    print('Hello World!')
```

```
# call the function
```

```
greet()
```

```
print('Outside function')
```

Output :-

```
Hello World!
```

```
Outside function
```

Example 2 :-

```
# function with two arguments
```

```
def add_numbers(num1, num2):
```

```
sum = num1 + num2

print("Sum: ",sum)

# function call with two values

add_numbers(5, 4)

# Output: Sum: 9
```

3. Explain function without arguments.

The function without any arguments is a function doesn't have any arguments and doesn't return any values.

Example :-

```
def greet():

    print('Hello World!')

# call the function

greet()

print('Outside function')
```

Output :-

```
Hello World!

Outside function
```

4. Explain return statement in function.

A return statement is used to end the execution of the function call and “returns” the result (value of the expression following the return keyword) to the caller. The statements after the return statements are not executed. If the return statement is without any expression, then the special value None is returned. A return statement is overall used to invoke a function so that the passed statements can be executed.

Note: Return statement can not be used outside the function.

Syntax:

```
def fun():

    statements
```

return [expression]

Example :-

```
def add(a, b):
```

```
# returning sum of a and b
```

```
    return a + b
```

```
res = add(2, 3)
```

```
print (" The result of add function = ",res)
```

Output :-

The result of add function = 5

6. What is recursion ? Explain with example.

The term Recursion can be defined as the process of defining something in terms of itself. In simple words, it is a process in which a function calls itself directly or indirectly.

Syntax:

```
def func(): <--
```

```
    |
```

```
    | (recursive call)
```

```
    |
```

```
func() ----
```

Advantages of using recursion

- A complicated function can be split down into smaller sub-problems utilizing recursion.
- Sequence creation is simpler through recursion than utilizing any nested iteration.
- Recursive functions render the code look simple and effective.

Disadvantages of using recursion

- A lot of memory and time is taken through recursive calls which makes it expensive for use.
- Recursive functions are challenging to debug.
- The reasoning behind recursion can sometimes be tough to think through.

Example :-

```
def factorial(x):  
    """This is a recursive function to find the factorial of an integer"""  
    if x == 1:  
        return 1  
    else:  
        return (x * factorial(x-1))  
num = int(input("enter any no ="))  
print("The factorial of", num, "is", factorial(num))
```

Output :-

enter any no =5

The factorial of 5 is 120

7. Explain arbitrary function with example.

Sometimes, we do not know in advance the number of arguments that will be passed into a function. To handle this kind of situation, we can use arbitrary arguments in Python.

Arbitrary arguments allow us to pass a varying number of values during a function call.

We use an asterisk (*) before the parameter name to denote this kind of argument.

Example :-

program to find sum of multiple numbers

```
def find_sum(*numbers):
```

```
    result = 0
```

```
    for num in numbers:
```

```
        result = result + num
```

```
    print("Sum = ", result)
```

function call with 3 arguments

```
find_sum(1, 2, 3)
```

function call with 2 arguments

find_sum(4, 9)

Output :-

Sum = 6

Sum = 13